

CSP 开卷速查与模板包

目录

C++17 代码模板全集	2
base	2
bfs_graph	2
bfs_grid	4
big_sim_command_parser	5
big_sim_state_machine	6
binary_search	8
bitmask_dp	9
brute_force_intervals	10
brute_force_permutation	11
brute_force_subsets	12
difference_1d	13
difference_2d	14
dijkstra	15
dsu	17
fast_io	18
fenwick	21
floyd	23
gcd_fastpow_comb	24
interval_dp	26
kmp	27
knapsack_01	29
knapsack_complete	30
kruskal	31
prefix_sum_1d	32
prefix_sum_2d	33
segment_tree	35
string_hash	37
stringstream_parser	38
topo_sort	39
vector_map_set	41

C++17 代码模板全集

页眉：模块名 = 代码模板合集 | 页码 = ____ | 触发词 = 模板/照抄

翻页索引：按文件名顺序

base

```
/*
【模板名】C++17 最小模板
页眉：模块名 = 基础模板 / 页码 = ____ / 触发词 = 空白/最小程序
题面触发词：任何题的起手模板
适用数据范围：通用
时间复杂度： $O(1)$ 
空间复杂度： $O(1)$ 
你只需要改这里：
- 读入与输出逻辑
常见错误：
- 忘记加同步关闭
最小样例：
输入：
1
输出：
1
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // TODO: 读入数据 (cin >> n >> ...)
    // TODO: 计算答案
    // TODO: 输出结果 (cout << ans << "\n")
    return 0;
}
```

bfs_graph

```
/*
【模板名】图 BFS（最短路）
页眉：模块名 = BFS / 页码 = ____ / 触发词 = 无权图/最短路
题面触发词：无权图最短路、最少步数
适用数据范围： $n \leq 2e5$ ,  $m \leq 2e5$ 
```

时间复杂度: $O(n+m)$

空间复杂度: $O(n+m)$

你只需要改这里:

- 起点与输出

常见错误:

- 忘记初始化 *dist*

最小样例:

输入:

4 3 1

1 2

2 3

3 4

输出:

0 1 2 3

*/

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
using ll = long long;
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, s;
    if (!(cin >> n >> m >> s)) return 0;
    // 改这里: 起点 s
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        // 无向图: 双向加边
        g[u].push_back(v);
        g[v].push_back(u);
    }
    // dist = -1 表示未访问
    vector<int> dist(n + 1, -1);
    queue<int> q;
    q.push(s);
    dist[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        // 扩展当前点的所有邻居
        for (int v : g[u]) if (dist[v] == -1) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}
```

```

    for (int i = 1; i <= n; i++) {
        if (i > 1) cout << " ";
        cout << dist[i];
    }
    cout << "\n";
    return 0;
}

```

bfs_grid

/*

【模板名】网格 BFS（最短步数）

页眉：模块名 = BFS / 页码 = ____ / 触发词 = 网格/最短路/等权

题面触发词：网格最短步数、每步代价相同

适用数据范围： $n, m \leq 2000$

时间复杂度： $O(nm)$

空间复杂度： $O(nm)$

你只需要改这里：

- 起点终点字符

- 4/8 方向

常见错误：

- 忘记标记访问

最小样例：

输入：

3 3

S..

.#. .

..T

输出：

4

*/

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
using ll = long long;
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    // 改这里：网格行列大小
    // 读入网格
    vector<string> g(n);
    for (int i = 0; i < n; i++) cin >> g[i];
    queue<pair<int,int>> q;

```

```

// dist = -1 表示未访问
vector<vector<int>>> dist(n, vector<int>(m, -1));
int sx = -1, sy = -1, tx = -1, ty = -1;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        // 找起点/终点
        if (g[i][j] == 'S') { sx = i; sy = j; }
        if (g[i][j] == 'T') { tx = i; ty = j; }
    }
}
// 从起点开始 BFS
q.push({sx, sy});
dist[sx][sy] = 0;
// 改这里: 4 方向或 8 方向
int dx[4] = {1, -1, 0, 0};
int dy[4] = {0, 0, 1, -1};
while (!q.empty()) {
    auto [x, y] = q.front(); q.pop();
    for (int k = 0; k < 4; k++) {
        int nx = x + dx[k], ny = y + dy[k];
        // 边界和墙判断
        if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
        if (g[nx][ny] == '#') continue;
        if (dist[nx][ny] != -1) continue;
        dist[nx][ny] = dist[x][y] + 1;
        q.push({nx, ny});
    }
}
// 若不可达, dist 为 -1
cout << dist[tx][ty] << "\n";
return 0;
}

```

big_sim_command_parser

```

/*
【模板名】大模拟命令解析
页眉: 模块名 = 大模拟 / 页码 = ____ / 触发词 = 命令/解析
题面触发词: 命令、日志、含空格字段
适用数据范围: 命令数  $\leq 2e5$ 
时间复杂度:  $O(\text{总字符数})$ 
空间复杂度:  $O(1)$ 
你只需要改这里:
- 命令分发逻辑
常见错误:
- 忘记读取上一行换行

```

最小样例:

输入:

2

add user1 3

del user1

输出:

2

*/

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
using ll = long long;
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    // 改这里: 命令数量 q
    string line;
    getline(cin, line); // 吃掉上一行换行

    int count_add = 0;
    for (int i = 0; i < q; i++) {
        getline(cin, line);
        // 用 stringstream 拆分整行命令
        stringstream ss(line);
        string cmd; ss >> cmd;
        if (cmd == "add") count_add++;
        // 改这里: 按命令处理 (分支/参数解析)
    }
    cout << count_add << "\n";
    return 0;
}
```

big_sim__state__machine

/*

【模板名】状态机模板

页眉: 模块名 = 大模拟 / 页码 = ____ / 触发词 = 状态/切换

题面触发词: 状态切换、规则驱动

适用数据范围: 命令数 $\leq 2e5$

时间复杂度: $O(q)$

空间复杂度: $O(1)$

你只需要改这里:

- 状态枚举

- 转移规则

常见错误:

- 忘记非法状态判断

最小样例:

输入:

4

OPEN

LOCK

UNLOCK

CLOSE

输出:

OK

DENY

OK

OK

*/

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
using ll = long long;
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    const int CLOSED = 0, OPEN = 1, LOCKED = 2;
    // 初始状态
    int state = CLOSED;
    while (q--) {
        string cmd; cin >> cmd;
        // 根据命令进行状态转移
        if (cmd == "OPEN") {
            if (state == CLOSED) { state = OPEN; cout << "OK\n"; }
            else cout << "DENY\n";
        } else if (cmd == "CLOSE") {
            if (state == OPEN) { state = CLOSED; cout << "OK\n"; }
            else cout << "DENY\n";
        } else if (cmd == "LOCK") {
            if (state == OPEN) { state = LOCKED; cout << "OK\n"; }
            else cout << "DENY\n";
        } else if (cmd == "UNLOCK") {
            if (state == LOCKED) { state = OPEN; cout << "OK\n"; }
            else cout << "DENY\n";
        }
    }
}
```

```

    return 0;
}

```

binary__search

```

/*
【模板名】二分查找（第一个  $\geq target$ ）
页眉：模块名 = 二分 / 页码 = ____ / 触发词 = 答案单调/最小满足
题面触发词：最小满足条件、答案单调
适用数据范围： $n \leq 2e5$ 
时间复杂度： $O(\log n)$ 
空间复杂度： $O(1)$ 
你只需要改这里：
- check(mid) 条件
- 输出格式
常见错误：
- 边界死循环
最小样例：
输入：
5 4
1 2 4 4 9
输出：
3
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    ll target;
    if (!(cin >> n >> target)) return 0;
    // 改这里：target 为目标值
    vector<ll> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];
    // 数组必须是有序的（升序）
    int l = 0, r = n - 1;
    while (l < r) {
        int mid = l + (r - l) / 2;
        // 找第一个  $\geq target$  的位置
        if (a[mid] >= target) r = mid;
        else l = mid + 1;
    }
}

```



```

    // 未找到时输出 -1
    if (n == 0 || a[1] < target) cout << -1 << "\n";
    else cout << (1 + 1) << "\n";
    return 0;
}

```

bitmask_dp

```

/*
【模板名】状压 DP（旅行商简版）
页眉：模块名 = 状压 DP / 页码 = ____ / 触发词 =  $n \leq 20$  / 子集
题面触发词：  $n \leq 20$ 、状态包含已选集合
适用数据范围：  $n \leq 20$ 
时间复杂度：  $O(n^2 * 2^n)$ 
空间复杂度：  $O(n * 2^n)$ 
你只需要改这里：
- 起点与终点规则
常见错误：
- mask 下标溢出
最小样例：
输入：
3
0 1 2
1 0 3
2 3 0
输出：
3
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    // 改这里：  $n \leq 20$  才能用状压
    vector<vector<ll>> w(n, vector<ll>(n));
    for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) cin >> w[i][j];

    const ll INF = (1LL << 60);
    int N = 1 << n;
    // dp[mask][u]: 走到 u, 已访问集合为 mask 的最小代价
    vector<vector<ll>> dp(N, vector<ll>(n, INF));

```

```

// 从 0 号点出发
dp[1][0] = 0;
for (int mask = 1; mask < N; mask++) {
    for (int u = 0; u < n; u++) {
        if (!(mask & (1 << u))) continue;
        if (dp[mask][u] >= INF / 2) continue;
        for (int v = 0; v < n; v++) {
            if (mask & (1 << v)) continue;
            int nmask = mask | (1 << v);
            // 从 u 走到 v
            dp[nmask][v] = min(dp[nmask][v], dp[mask][u] + w[u][v]);
        }
    }
}
ll ans = INF;
for (int i = 0; i < n; i++) ans = min(ans, dp[N - 1][i]);
// 改这里: 是否需要回到起点
cout << ans << "\n";
return 0;
}

```

brute_force_intervals

```

/*
【模板名】枚举区间 ( $O(n^2)$ )
页眉: 模块名 = 暴力 / 页码 = ____ / 触发词 = 枚举区间
题面触发词:  $n \leq 2000$ , 区间枚举
适用数据范围:  $n \leq 2000$ 
时间复杂度:  $O(n^2)$ 
空间复杂度:  $O(1)$ 
你只需要改这里:
- 统计逻辑
常见错误:
- 双重循环边界
最小样例:
输入:
4
1 -2 3 4
输出:
7
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {

```

```

ios::sync_with_stdio(false);
cin.tie(nullptr);

int n;
if (!(cin >> n)) return 0;
// 改这里:  $n \leq 2000$  才能  $O(n^2)$ 
vector<ll> a(n + 1);
for (int i = 1; i <= n; i++) cin >> a[i];

ll best = LLONG_MIN / 4;
// 枚举所有区间  $[l, r]$ 
for (int l = 1; l <= n; l++) {
    ll sum = 0;
    for (int r = l; r <= n; r++) {
        sum += a[r];
        best = max(best, sum);
    }
}
// 改这里: 输出最大值/最小值/计数
cout << best << "\n";
return 0;
}

```

brute_force_permutation

```

/*
【模板名】全排列枚举
页眉: 模块名 = 暴力 / 页码 = ____ / 触发词 = 全排列/ $n \leq 10$ 
题面触发词:  $n \leq 10$ , 枚举全排列
适用数据范围:  $n \leq 10$ 
时间复杂度:  $O(n! * n)$ 
空间复杂度:  $O(n)$ 
你只需要改这里:
- 检查逻辑
常见错误:
- 忘记初始化序列
最小样例:
输入:
3
输出:
6
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    // 改这里:  $n \leq 10$  才能枚举全排列
    vector<int> p(n);
    // 从  $1..n$  的初始排列开始
    iota(p.begin(), p.end(), 1);
    int cnt = 0;
    do {
        cnt++;
        // 改这里: 检查当前排列
    } while (next_permutation(p.begin(), p.end()));
    cout << cnt << "\n";
    return 0;
}

```

brute_force_subsets

```

/*
【模板名】子集枚举（位运算）
页眉：模块名 = 暴力 / 页码 = ____ / 触发词 = 子集/状压
题面触发词：  $n \leq 20$ ，枚举子集
适用数据范围：  $n \leq 20$ 
时间复杂度：  $O(n \cdot 2^n)$ 
空间复杂度：  $O(1)$ 
你只需要改这里：
- 统计逻辑
常见错误：
- 位运算顺序
最小样例：
输入：
3
1 2 3
输出：
6
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

```

```

int n;
if (!(cin >> n)) return 0;
// 改这里: n<=20 才能枚举
vector<ll> a(n);
for (int i = 0; i < n; i++) cin >> a[i];

ll best = 0;
int N = 1 << n;
// mask 的每一位表示是否选第 i 个元素
for (int mask = 0; mask < N; mask++) {
    ll sum = 0;
    for (int i = 0; i < n; i++) {
        if (mask & (1 << i)) sum += a[i];
    }
    best = max(best, sum);
}
// 改这里: 输出最大值/计数/是否存在
cout << best << "\n";
return 0;
}

```

difference_1d

```

/*
【模板名】差分 1D (区间加)
页眉: 模块名 = 差分 / 页码 = ____ / 触发词 = 多次区间加
题面触发词: 多次区间加、最后输出数组
适用数据范围:  $n \leq 1e6, q \leq 1e6$ 
时间复杂度:  $O(n+q)$ 
空间复杂度:  $O(n)$ 
你只需要改这里:
- 更新规则
常见错误:
-  $r+1$  越界
最小样例:
输入:
5 2
1 2 3 4 5
1 3 10
2 5 -2
输出:
11 10 11 2 3
*/
#include <bits/stdc++.h>
using namespace std;

```

```

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    // 改这里: n 为数组长度, q 为修改次数
    vector<ll> a(n + 1), diff(n + 2, 0);
    for (int i = 1; i <= n; i++) cin >> a[i];
    // 构建差分数组: diff[i] = a[i] - a[i-1]
    for (int i = 1; i <= n; i++) {
        diff[i] += a[i] - a[i - 1];
    }
    while (q--) {
        int l, r; ll v;
        cin >> l >> r >> v;
        // 区间加: 在 l 加 v, 在 r+1 减 v
        diff[l] += v;
        diff[r + 1] -= v;
    }
    // 差分前缀还原原数组
    for (int i = 1; i <= n; i++) {
        diff[i] += diff[i - 1];
        if (i > 1) cout << " ";
        cout << diff[i];
    }
    cout << "\n";
    return 0;
}

```

difference_2d

/*
【模板名】差分 2D (子矩阵加)
 页眉: 模块名 = 差分 / 页码 = ____ / 触发词 = 多次子矩阵加
 题面触发词: 多次矩形加、最后输出矩阵
 适用数据范围: $n, m \leq 2000$, $q \leq 2e5$
 时间复杂度: $O(nm+q)$
 空间复杂度: $O(nm)$
 你只需要改这里:
 - 更新规则
 常见错误:
 - 边界 $r+1/c+1$ 越界
 最小样例:

输入:

```
2 2 1
1 1 2 2 5
```

输出:

```
5 5
5 5
```

*/

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
using ll = long long;
```

```
int main() {
```

```
    ios::sync_with_stdio(false);
```

```
    cin.tie(nullptr);
```

```
    int n, m, q;
```

```
    if (!(cin >> n >> m >> q)) return 0;
```

```
    // 改这里: 矩阵大小和修改次数
```

```
    vector<vector<ll>> diff(n + 2, vector<ll>(m + 2, 0));
```

```
    while (q--) {
```

```
        int x1, y1, x2, y2; ll v;
```

```
        cin >> x1 >> y1 >> x2 >> y2 >> v;
```

```
        // 二维差分: 四角加减
```

```
        diff[x1][y1] += v;
```

```
        diff[x2 + 1][y1] -= v;
```

```
        diff[x1][y2 + 1] -= v;
```

```
        diff[x2 + 1][y2 + 1] += v;
```

```
    }
```

```
    for (int i = 1; i <= n; i++) {
```

```
        for (int j = 1; j <= m; j++) {
```

```
            // 二维前缀还原
```

```
            diff[i][j] = diff[i][j] + diff[i - 1][j] + diff[i][j - 1] - diff[i - 1][j - 1];
```

```
            if (j > 1) cout << " ";
```

```
            cout << diff[i][j];
```

```
        }
```

```
        cout << "\n";
```

```
    }
```

```
    return 0;
```

```
}
```

dijkstra

/*

【模板名】*Dijkstra* (非负边最短路)

页眉: 模块名 = 最短路 / 页码 = ____ / 触发词 = 非负边/最短路

题面触发词: 边权非负最短路

适用数据范围: $n \leq 2e5, m \leq 2e5$

时间复杂度: $O(m \log n)$

空间复杂度: $O(n+m)$

你只需要改这里:

- 是否有向

常见错误:

- 未使用 `long long`

最小样例:

输入:

3 3 1

1 2 2

2 3 3

1 3 10

输出:

0 2 5

*/

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
using ll = long long;
```

```
struct Edge {
```

```
    int to;
```

```
    ll w;
```

```
};
```

```
int main() {
```

```
    ios::sync_with_stdio(false);
```

```
    cin.tie(nullptr);
```

```
    int n, m, s;
```

```
    if (!(cin >> n >> m >> s)) return 0;
```

```
    // 改这里: 起点 s
```

```
    vector<vector<Edge>> g(n + 1);
```

```
    for (int i = 0; i < m; i++) {
```

```
        int u, v; ll w;
```

```
        cin >> u >> v >> w;
```

```
        // 无向图: 双向加边, 有向图则删掉一行
```

```
        g[u].push_back({v, w});
```

```
        g[v].push_back({u, w});
```

```
    }
```

```
    const ll INF = (1LL << 60);
```

```
    // dist 初始化为无穷大
```

```
    vector<ll> dist(n + 1, INF);
```

```
    // 小根堆: 每次取当前距离最小的点
```

```
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
```

```
    dist[s] = 0;
```



```

    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        // 过期的堆元素跳过
        if (d != dist[u]) continue;
        for (auto &e : g[u]) {
            // 松弛操作
            if (dist[e.to] > dist[u] + e.w) {
                dist[e.to] = dist[u] + e.w;
                pq.push({dist[e.to], e.to});
            }
        }
    }
    for (int i = 1; i <= n; i++) {
        if (i > 1) cout << " ";
        // 改这里: 不可达时输出什么
        cout << (dist[i] >= INF/2 ? -1 : dist[i]);
    }
    cout << "\n";
    return 0;
}

```

dsu

```

/*
【模板名】并查集 (DSU)
页眉: 模块名 = 并查集 / 页码 = ____ / 触发词 = 连通性/合并
题面触发词: 连通性、合并集合
适用数据范围:  $n \leq 2e5$ 
时间复杂度: 近似  $O(1)$ 
空间复杂度:  $O(n)$ 
你只需要改这里:
- 操作类型
常见错误:
- 忘记路径压缩
最小样例:
输入:
5 3
1 1 2
1 2 3
2 1 3
输出:
YES
*/
#include <bits/stdc++.h>
using namespace std;

```

```

using ll = long long;

struct DSU {
    vector<int> p, r;
    DSU(int n = 0) { init(n); }
    void init(int n) {
        p.resize(n + 1);
        r.assign(n + 1, 0);
        iota(p.begin(), p.end(), 0);
    }
    // 路径压缩
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    // 按秩合并
    void unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return;
        if (r[a] < r[b]) swap(a, b);
        p[b] = a;
        if (r[a] == r[b]) r[a]++;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    // 改这里: 操作类型含义 (1 合并 / 2 查询)
    DSU dsu(n);
    while (q--) {
        int op, a, b;
        cin >> op >> a >> b;
        // op=1 合并, op=2 查询
        if (op == 1) dsu.unite(a, b);
        else cout << (dsu.find(a) == dsu.find(b) ? "YES" : "NO") << "\n";
    }
    return 0;
}

```

fast_io

/*

【模板名】快读快写

页眉: 模块名 = 基础模板 / 页码 = ____ / 触发词 = 大输入/卡常

题面触发词: 输入很大、IO 成为瓶颈

适用数据范围：数据总量 $\geq 1e6$

时间复杂度： $O(n)$

空间复杂度： $O(1)$

你只需要改这里：

- 读入类型与输出内容

常见错误：

- 忘记处理负数

最小样例：

输入：

3

1 2 3

输出：

6

*/

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
using ll = long long;
```

```
struct FastScanner {
    static const int BUFSIZE = 1 << 20;
    int idx, size;
    char buf[BUFSIZE];
    FastScanner() : idx(0), size(0) {}

```

// 从缓冲区读取一个字符

```
inline char read() {
    if (idx >= size) {
        size = (int)fread(buf, 1, BUFSIZE, stdin);
        idx = 0;
        if (size == 0) return 0;
    }
    return buf[idx++];
}
```

```
bool nextLong(ll &out) {
    char c = read();
    if (!c) return false;
    // 跳过非数字字符
    while (c != '-' && (c < '0' || c > '9')) {
        c = read();
        if (!c) return false;
    }
    ll sign = 1;
    if (c == '-') { sign = -1; c = read(); }
    ll val = 0;
    // 读取连续数字
```

```

        while (c >= '0' && c <= '9') {
            val = val * 10 + (c - '0');
            c = read();
        }
        out = val * sign;
        return true;
    }

    // 读取一个不含空格的字符串
    bool nextString(string &s) {
        char c = read();
        if (!c) return false;
        while (isspace((unsigned char)c)) {
            c = read();
            if (!c) return false;
        }
        s.clear();
        while (c && !isspace((unsigned char)c)) {
            s.push_back(c);
            c = read();
        }
        return true;
    }
};

struct FastOutput {
    static const int BUFSIZE = 1 << 20;
    int idx;
    char buf[BUFSIZE];
    FastOutput() : idx(0) {}
    ~FastOutput() { flush(); }

    // 刷新输出缓冲区
    void flush() {
        if (idx) {
            fwrite(buf, 1, idx, stdout);
            idx = 0;
        }
    }

    void pushChar(char c) {
        if (idx >= BUFSIZE) flush();
        buf[idx++] = c;
    }

    // 输出一个 Long Long

```

```

void writeLong(ll x, char end = '\n') {
    if (x == 0) {
        pushChar('0');
        pushChar(end);
        return;
    }
    if (x < 0) { pushChar('-'); x = -x; }
    char s[32];
    int n = 0;
    while (x > 0) { s[n++] = char('0' + x % 10); x /= 10; }
    for (int i = n - 1; i >= 0; i--) pushChar(s[i]);
    pushChar(end);
}

};

int main() {
    FastScanner fs;
    FastOutput fo;

    ll n;
    if (!fs.nextLong(n)) return 0;
    ll sum = 0;
    for (ll i = 0; i < n; i++) {
        ll x;
        fs.nextLong(x);
        sum += x;
    }
    // 输出计算结果
    fo.writeLong(sum);
    return 0;
}

```

fenwick

/*

【模板名】树状数组（单点加/前缀和）

页眉：模块名 = 树状数组 / 页码 = ____ / 触发词 = 区间和/单点加

题面触发词：单点修改、区间查询

适用数据范围： $n \leq 2e5$

时间复杂度： $O(\log n)$

空间复杂度： $O(n)$

你只需要改这里：

- 操作类型

常见错误：

- 下标从 1 开始

最小样例：

输入:

```
5 3
1 2 3 4 5
2 1 3
1 2 5
2 1 3
```

输出:

```
6
11
```

*/

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
using ll = long long;
```

```
struct Fenwick {
```

```
    int n;
```

```
    vector<ll> bit;
```

```
    Fenwick(int n = 0) { init(n); }
```

```
    void init(int n_) { n = n_; bit.assign(n + 1, 0); }
```

```
    void add(int idx, ll val) {
```

```
        // 单点加: 往上更新
```

```
        for (; idx <= n; idx += idx & -idx) bit[idx] += val;
```

```
    }
```

```
    ll sumPrefix(int idx) {
```

```
        ll res = 0;
```

```
        // 前缀和: 往上跳
```

```
        for (; idx > 0; idx -= idx & -idx) res += bit[idx];
```

```
        return res;
```

```
    }
```

```
    ll rangeSum(int l, int r) { return sumPrefix(r) - sumPrefix(l - 1); }
```

```
};
```

```
int main() {
```

```
    ios::sync_with_stdio(false);
```

```
    cin.tie(nullptr);
```

```
    int n, q;
```

```
    if (!(cin >> n >> q)) return 0;
```

```
    // 改这里: 数组大小 n, 操作数量 q
```

```
    Fenwick fw(n);
```

```
    for (int i = 1; i <= n; i++) {
```

```
        ll x; cin >> x;
```

```
        // 建树: 逐点加入
```

```
        fw.add(i, x);
```

```
    }
```

```
    while (q--) {
```

```

    int op; cin >> op;
    // 改这里: 操作类型定义
    if (op == 1) {
        int idx; ll delta; cin >> idx >> delta;
        // 单点加
        fw.add(idx, delta);
    } else {
        int l, r; cin >> l >> r;
        // 区间和
        cout << fw.rangeSum(l, r) << "\n";
    }
}
return 0;
}

```

floyd

```

/*
【模板名】Floyd (全点对最短路)
页眉: 模块名 = 最短路 / 页码 = ____ / 触发词 = 全点对/小  $n$ 
题面触发词: 所有点对最短路、 $n$  小
适用数据范围:  $n \leq 500$ 
时间复杂度:  $O(n^3)$ 
空间复杂度:  $O(n^2)$ 
你只需要改这里:
- 输出格式
常见错误:
-  $INF$  溢出
最小样例:
输入:
3 3
1 2 2
2 3 3
1 3 10
输出:
0 2 5
2 0 3
5 3 0
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

```

```

int n, m;
if (!(cin >> n >> m)) return 0;
// 改这里: 节点数量和边数量
const ll INF = (1LL << 60);
// dist 初始化为 INF, 自己到自己为 0
vector<vector<ll>> dist(n + 1, vector<ll>(n + 1, INF));
for (int i = 1; i <= n; i++) dist[i][i] = 0;
for (int i = 0; i < m; i++) {
    int u, v; ll w;
    cin >> u >> v >> w;
    // 可能有重边, 取最小
    dist[u][v] = min(dist[u][v], w);
    dist[v][u] = min(dist[v][u], w);
}
// 三重循环: 枚举中转点 k
for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j])
                dist[i][j] = dist[i][k] + dist[k][j];
        }
    }
}
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (j > 1) cout << " ";
        // 改这里: 不可达的输出
        cout << dist[i][j];
    }
    cout << "\n";
}
return 0;
}

```

gcd_fastpow_comb

/*

【模板名】gcd/lcm/快速幂/组合数

页眉: 模块名 = 数学 / 页码 = ____ / 触发词 = gcd/快速幂/组合数

题面触发词: 最大公约数、最小公倍数、幂、组合数

适用数据范围: int64, 组合数需模为质数

时间复杂度: $O(\log n)$ 或 $O(n)$

空间复杂度: $O(n)$

你只需要改这里:

- 模数 mod

- 组合数上限

常见错误:

- `mod` 不是质数仍用费马逆元

最小样例:

输入:

```
5 2 1000000007
```

输出:

```
10
```

```
*/
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
using ll = long long;
```

```
ll gcdll(ll a, ll b) { return b ? gcdll(b, a % b) : a; }
```

```
ll lcmll(ll a, ll b) { return a / gcdll(a, b) * b; }
```

```
ll modpow(ll a, ll e, ll mod) {
```

```
    // 快速幂:  $a^e \% mod$ 
```

```
    ll r = 1 % mod;
```

```
    a %= mod;
```

```
    while (e > 0) {
```

```
        if (e & 1) r = (__int128)r * a % mod;
```

```
        a = (__int128)a * a % mod;
```

```
        e >>= 1;
```

```
    }
```

```
    return r;
```

```
}
```

```
struct Comb {
```

```
    ll mod;
```

```
    vector<ll> fact, invfact;
```

```
    void init(int n, ll m) {
```

```
        mod = m;
```

```
        fact.assign(n + 1, 1);
```

```
        invfact.assign(n + 1, 1);
```

```
        // 阶乘
```

```
        for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i % mod;
```

```
        // 费马逆元 (要求 mod 为质数)
```

```
        invfact[n] = modpow(fact[n], mod - 2, mod);
```

```
        for (int i = n; i >= 1; i--) invfact[i - 1] = invfact[i] * i % mod;
```

```
    }
```

```
    ll C(int n, int k) {
```

```
        if (k < 0 || k > n) return 0;
```

```
        // 组合数  $C(n, k)$ 
```

```
        return fact[n] * invfact[k] % mod * invfact[n - k] % mod;
```

```
    }
```

```
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    ll mod;
    if (!(cin >> n >> k >> mod)) return 0;
    // 改这里:  $n$  为上限,  $k$  为选取数,  $mod$  需为质数
    Comb comb;
    comb.init(n, mod);
    cout << comb.C(n, k) << "\n";
    return 0;
}
```

interval_dp

```
/*
【模板名】区间 DP（合并最小代价）
页眉：模块名 = 区间 DP / 页码 = ____ / 触发词 = 区间合并
题面触发词：合并区间、最小代价
适用数据范围： $n \leq 500$ 
时间复杂度： $O(n^3)$ 
空间复杂度： $O(n^2)$ 
你只需要改这里：
- 代价计算
常见错误：
-  $dp$  初始化为  $INF$ 
最小样例：
输入：
4
1 2 3 4
输出：
19
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
```

```

// 改这里:  $n$  表示区间元素数量
vector<ll> a(n + 1), pre(n + 1, 0);
for (int i = 1; i <= n; i++) {
    cin >> a[i];
    pre[i] = pre[i - 1] + a[i];
}
//  $sum(l, r)$  = 区间和, 用于合并代价
auto sum = [&](int l, int r) { return pre[r] - pre[l - 1]; };

const ll INF = (1LL << 60);
vector<vector<ll>> dp(n + 1, vector<ll>(n + 1, 0));
// 按区间长度递推
for (int len = 2; len <= n; len++) {
    for (int l = 1; l + len - 1 <= n; l++) {
        int r = l + len - 1;
        dp[l][r] = INF;
        // 枚举分割点  $k$ 
        for (int k = l; k < r; k++) {
            // 改这里: 合并代价可替换
            dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r] + sum(l, r));
        }
    }
}
cout << dp[1][n] << "\n";
return 0;
}

```

kmp

```

/*
【模板名】KMP 字符串匹配
页眉: 模块名 = 字符串 / 页码 = ____ / 触发词 = 模式匹配
题面触发词: 子串匹配、模式匹配
适用数据范围:  $|s| \leq 1e6$ 
时间复杂度:  $O(n+m)$ 
空间复杂度:  $O(m)$ 
你只需要改这里:
- 输出位置格式
常见错误:
-  $next$  数组下标错
最小样例:
输入:
ababab
abab
输出:
1 3

```

```

*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

vector<int> build_next(const string &p) {
    int m = (int)p.size();
    vector<int> ne(m, 0);
    // ne[i]: p[0..i] 的最长相等前后缀长度
    for (int i = 1, j = 0; i < m; i++) {
        while (j > 0 && p[i] != p[j]) j = ne[j - 1];
        if (p[i] == p[j]) j++;
        ne[i] = j;
    }
    return ne;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s, p;
    if (!(cin >> s >> p)) return 0;
    // 改这里: s= 文本, p= 模式串
    auto ne = build_next(p);
    vector<int> pos;
    for (int i = 0, j = 0; i < (int)s.size(); i++) {
        // 失配回退
        while (j > 0 && s[i] != p[j]) j = ne[j - 1];
        if (s[i] == p[j]) j++;
        if (j == (int)p.size()) {
            // 找到一次匹配
            pos.push_back(i - (int)p.size() + 2); // 1-based
            j = ne[j - 1];
        }
    }
    for (int i = 0; i < (int)pos.size(); i++) {
        if (i) cout << " ";
        // 改这里: 输出 1-based 或 0-based
        cout << pos[i];
    }
    cout << "\n";
    return 0;
}

```

knapsack_01

```

/*
【模板名】0/1 背包
页眉：模块名 = 背包 / 页码 = ____ / 触发词 = 每件最多一次
题面触发词：每件物品只能选 0/1 次
适用数据范围： $n \leq 2e5$ ,  $w \leq 2e5$ 
时间复杂度： $O(nW)$ 
空间复杂度： $O(W)$ 
你只需要改这里：
- 价值与容量
常见错误：
- 容量遍历方向写错（必须倒序）
最小样例：
输入：
3 5
2 3
3 4
4 5
输出：
7
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, W;
    if (!(cin >> n >> W)) return 0;
    // 改这里：物品数量  $n$ 、容量  $W$ 
    vector<ll> w(n), v(n);
    for (int i = 0; i < n; i++) cin >> w[i] >> v[i];
    //  $dp[j]$  表示容量为  $j$  的最大价值
    vector<ll> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        // 0/1 背包必须倒序遍历容量
        for (int j = W; j >= w[i]; j--) {
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
        }
    }
    // 改这里：输出  $dp[W]$  或最大值
    cout << dp[W] << "\n";
    return 0;
}

```

```
}
```

knapsack__complete

```
/*
【模板名】完全背包
页眉：模块名 = 背包 / 页码 = ____ / 触发词 = 每件无限次
题面触发词：每件物品可选多次
适用数据范围： $n \leq 2e5$ ,  $W \leq 2e5$ 
时间复杂度： $O(nW)$ 
空间复杂度： $O(W)$ 
你只需要改这里：
- 价值与容量
常见错误：
- 容量遍历方向写错（必须正序）
最小样例：
输入：
2 5
2 3
3 4
输出：
7
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, W;
    if (!(cin >> n >> W)) return 0;
    // 改这里：物品数量  $n$ 、容量  $W$ 
    vector<ll> w(n), v(n);
    for (int i = 0; i < n; i++) cin >> w[i] >> v[i];
    //  $dp[j]$  表示容量为  $j$  的最大价值
    vector<ll> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        // 完全背包必须正序遍历容量
        for (int j = w[i]; j <= W; j++) {
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
        }
    }
    // 改这里：输出  $dp[W]$  或最大值
    cout << dp[W] << "\n";
}
```

```

    return 0;
}

```

kruskal

```

/*
【模板名】Kruskal（最小生成树）
页眉：模块名 = 最小生成树 / 页码 = ____ / 触发词 = 最小生成树
题面触发词：最小生成树、连通最小代价
适用数据范围： $n \leq 2e5, m \leq 2e5$ 
时间复杂度： $O(m \log m)$ 
空间复杂度： $O(n+m)$ 
你只需要改这里：
- 是否无向
常见错误：
- 图不连通
最小样例：
输入：
3 3
1 2 1
2 3 2
1 3 10
输出：
3
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct DSU {
    vector<int> p, r;
    DSU(int n = 0) { init(n); }
    void init(int n) { p.resize(n + 1); r.assign(n + 1, 0); iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (r[a] < r[b]) swap(a, b);
        p[b] = a;
        if (r[a] == r[b]) r[a]++;
        return true;
    }
};

struct Edge {
    int u, v;

```

```

    ll w;
    bool operator<(const Edge &other) const { return w < other.w; }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    // 改这里: 若是有向图需先转无向
    vector<Edge> edges(m);
    for (int i = 0; i < m; i++) {
        cin >> edges[i].u >> edges[i].v >> edges[i].w;
    }
    // 按边权排序
    sort(edges.begin(), edges.end());
    DSU dsu(n);
    ll total = 0;
    int used = 0;
    for (auto &e : edges) {
        // 能连通就选这条边
        if (dsu.unite(e.u, e.v)) {
            total += e.w;
            used++;
        }
    }
    // 若未连通则无生成树
    if (used != n - 1) cout << -1 << "\n";
    else cout << total << "\n";
    return 0;
}

```

prefix_sum_1d

/*
【模板名】前缀和 1D
 页眉: 模块名 = 前缀和 / 页码 = ____ / 触发词 = 区间和/子段和
 题面触发词: 区间和、连续子段和
 适用数据范围: $n \leq 1e6, q \leq 1e6$
 时间复杂度: 预处理 $O(n)$, 查询 $O(1)$
 空间复杂度: $O(n)$
 你只需要改这里:
 - 输入格式
 - 查询输出
 常见错误:

- 下标从 1 还是 0

最小样例:

输入:

```
5 3
1 2 3 4 5
1 3
2 5
4 4
```

输出:

```
6
14
4
```

*/

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
using ll = long long;
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    // 改这里: n 为数组长度, q 为查询数量
    // pre[i] 表示 a[1..i] 的和
    vector<ll> pre(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        ll x; cin >> x;
        pre[i] = pre[i - 1] + x;
    }
    while (q--) {
        int l, r; cin >> l >> r;
        // 区间和 = 前缀差
        cout << pre[r] - pre[l - 1] << "\n";
    }
    return 0;
}
```

prefix_sum__2d

/*

【模板名】前缀和 2D

页眉: 模块名 = 前缀和 / 页码 = ____ / 触发词 = 子矩阵和

题面触发词: 矩形区域和、子矩阵和

适用数据范围: $n, m \leq 2000$

时间复杂度: 预处理 $O(nm)$, 查询 $O(1)$

空间复杂度: $O(nm)$

你只需要改这里:

- 输入矩阵与查询

常见错误:

- 下标从 1 开始

最小样例:

输入:

```
2 2 2
1 2
3 4
1 1 2 2
1 2 2 2
```

输出:

```
10
6
```

*/

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
using ll = long long;
```

```
int main() {
```

```
    ios::sync_with_stdio(false);
```

```
    cin.tie(nullptr);
```

```
    int n, m, q;
```

```
    if (!(cin >> n >> m >> q)) return 0;
```

```
    // 改这里: n 行 m 列, q 次查询
```

```
    // pre[i][j] 表示子矩阵 (1,1)-(i,j) 的和
```

```
    vector<vector<ll>> pre(n + 1, vector<ll>(m + 1, 0));
```

```
    for (int i = 1; i <= n; i++) {
```

```
        for (int j = 1; j <= m; j++) {
```

```
            ll x; cin >> x;
```

```
            // 经典二维前缀和公式
```

```
            pre[i][j] = pre[i - 1][j] + pre[i][j - 1] - pre[i - 1][j - 1] + x;
```

```
        }
```

```
    }
```

```
    while (q--) {
```

```
        int x1, y1, x2, y2;
```

```
        cin >> x1 >> y1 >> x2 >> y2;
```

```
        // 子矩阵和 = 四块前缀相减
```

```
        ll ans = pre[x2][y2] - pre[x1 - 1][y2] - pre[x2][y1 - 1] + pre[x1 - 1][y1 - 1];
```

```
        cout << ans << "\n";
```

```
    }
```

```
    return 0;
```

```
}
```

segment__tree

```

/*
【模板名】线段树（区间加/区间和）
页眉：模块名 = 线段树 / 页码 = ____ / 触发词 = 区间修改/区间查询
题面触发词：区间修改、区间查询
适用数据范围： $n \leq 2e5$ 
时间复杂度： $O(\log n)$ 
空间复杂度： $O(n)$ 
你只需要改这里：
- 操作类型
常见错误：
- lazy 下推遗漏
最小样例：
输入：
5 3
1 2 3 4 5
2 1 3
1 2 4 2
2 1 3
输出：
6
10
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct SegTree {
    struct Node { ll sum = 0, lazy = 0; };
    int n;
    vector<Node> st;
    SegTree(int n = 0) { init(n); }
    void init(int n_) { n = n_; st.assign(4 * n + 4, {}); }

    void build(vector<ll> &a, int p, int l, int r) {
        if (l == r) { st[p].sum = a[l]; return; }
        int m = (l + r) / 2;
        // 递归建树
        build(a, p * 2, l, m);
        build(a, p * 2 + 1, m + 1, r);
        st[p].sum = st[p * 2].sum + st[p * 2 + 1].sum;
    }

    void apply(int p, int l, int r, ll val) {
        // 把懒标记作用在节点上

```

```

        st[p].sum += val * (r - l + 1);
        st[p].lazy += val;
    }

    void push(int p, int l, int r) {
        if (st[p].lazy == 0 || l == r) return;
        int m = (l + r) / 2;
        // 把懒标记下推给子节点
        apply(p * 2, l, m, st[p].lazy);
        apply(p * 2 + 1, m + 1, r, st[p].lazy);
        st[p].lazy = 0;
    }

    void rangeAdd(int p, int l, int r, int ql, int qr, ll val) {
        if (ql <= l && r <= qr) { apply(p, l, r, val); return; }
        push(p, l, r);
        int m = (l + r) / 2;
        if (ql <= m) rangeAdd(p * 2, l, m, ql, qr, val);
        if (qr > m) rangeAdd(p * 2 + 1, m + 1, r, ql, qr, val);
        // 回溯更新父节点
        st[p].sum = st[p * 2].sum + st[p * 2 + 1].sum;
    }

    ll rangeSum(int p, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return st[p].sum;
        push(p, l, r);
        int m = (l + r) / 2;
        ll res = 0;
        if (ql <= m) res += rangeSum(p * 2, l, m, ql, qr);
        if (qr > m) res += rangeSum(p * 2 + 1, m + 1, r, ql, qr);
        return res;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    // 改这里: 数组长度与操作数
    vector<ll> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    SegTree st(n);
    st.build(a, 1, 1, n);
}

```

```

while (q--) {
    int op; cin >> op;
    // 改这里: 操作类型定义
    if (op == 1) {
        int l, r; ll v; cin >> l >> r >> v;
        // 区间加
        st.rangeAdd(1, 1, n, l, r, v);
    } else {
        int l, r; cin >> l >> r;
        // 区间和
        cout << st.rangeSum(1, 1, n, l, r) << "\n";
    }
}
return 0;
}

```

string_hash

```

/*
【模板名】字符串哈希（滚动哈希）
页眉: 模块名 = 字符串 / 页码 = ____ / 触发词 = 子串相等/哈希
题面触发词: 比较子串相等
适用数据范围:  $|s| \leq 1e6, q \leq 2e5$ 
时间复杂度: 预处理  $O(n)$ , 查询  $O(1)$ 
空间复杂度:  $O(n)$ 
你只需要改这里:
- base/mod
常见错误:
- 溢出未取模
最小样例:
输入:
abcaab
2
1 3
3 5
输出:
294
588
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

```

```

const ll MOD = 1000000007LL;
const ll BASE = 911382323LL;

string s;
if (!(cin >> s)) return 0;
// 改这里: 字符串输入方式 (可能含空格)
int n = (int)s.size();
// h[i]: 前 i 个字符的哈希值, p[i]: BASE^i
vector<ll> h(n + 1, 0), p(n + 1, 1);
for (int i = 1; i <= n; i++) {
    h[i] = (h[i - 1] * BASE + s[i - 1]) % MOD;
    p[i] = (p[i - 1] * BASE) % MOD;
}
auto get = [&](int l, int r) {
    // 取子串 [l,r] 的哈希
    ll res = (h[r] - h[l - 1] * p[r - l + 1]) % MOD;
    if (res < 0) res += MOD;
    return res;
};

int q;
cin >> q;
while (q--) {
    int l, r; cin >> l >> r;
    // 改这里: l,r 是否 1-based
    cout << get(l, r) << "\n";
}
return 0;
}

```

stringstream_parser

/*

【模板名】*getline + stringstream* 解析

页眉: 模块名 = 字符串 / 页码 = ____ / 触发词 = 含空格/整行解析

题面触发词: 整行输入、含空格字段

适用数据范围: 行数 $\leq 2e5$

时间复杂度: $O(\text{总字符数})$

空间复杂度: $O(1)$

你只需要改这里:

- 字段类型和处理逻辑

常见错误:

- 忘记读取上一行换行

最小样例:

输入:

```

2
hello 1
world 2
输出:
2
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    // 改这里: 行数 n
    string line;
    getline(cin, line); // 吃掉上一行的换行

    int count_lines = 0;
    for (int i = 0; i < n; i++) {
        getline(cin, line);
        // 按空格拆分整行
        stringstream ss(line);
        string word;
        int num = 0;
        ss >> word >> num;
        if (!word.empty()) count_lines++;
    }
    cout << count_lines << "\n";
    return 0;
}

```

topo_sort

/*

【模板名】拓扑排序 (Kahn)

页眉: 模块名 = 拓扑排序 / 页码 = ____ / 触发词 = 依赖/先修

题面触发词: 依赖关系、先修顺序

适用数据范围: $n \leq 2e5$, $m \leq 2e5$

时间复杂度: $O(n+m)$

空间复杂度: $O(n+m)$

你只需要改这里:

- 输出格式

常见错误:

- 有环时未判断

最小样例:

输入:

3 2

1 2

2 3

输出:

1 2 3

*/

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
using ll = long long;
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    // 改这里: 读入依赖边 (u -> v)
    vector<vector<int>> g(n + 1);
    vector<int> indeg(n + 1, 0);
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        // u -> v
        g[u].push_back(v);
        indeg[v]++;
    }
    queue<int> q;
    // 入度为 0 的点先入队
    for (int i = 1; i <= n; i++) if (indeg[i] == 0) q.push(i);
    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : g[u]) {
            if (--indeg[v] == 0) q.push(v);
        }
    }
    // 有环时无法排出全部点
    if ((int)order.size() != n) {
        cout << "IMPOSSIBLE\n";
        return 0;
    }
    // 改这里: 输出拓扑序或其他格式
    for (int i = 0; i < n; i++) {
```



```

        if (i) cout << " ";
        cout << order[i];
    }
    cout << "\n";
    return 0;
}

```

vector__map__set

```

/*
【模板名】vector/map/set 速用
页眉：模块名 = 基础模板 / 页码 = ____ / 触发词 = 去重/统计/有序
题面触发词：去重、统计次数、有序输出
适用数据范围： $n \leq 2e5$ 
时间复杂度： $O(n \log n)$ 
空间复杂度： $O(n)$ 
你只需要改这里：
- 统计字段
- 输出格式
常见错误：
- map 遍历顺序误解
最小样例：
输入：
5
2 1 2 3 1
输出：
1 2
2 2
3 1
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    // 改这里：读入的元素类型/数量
    vector<ll> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    // 用 map 统计频次，map 会自动按 key 升序
    map<ll, ll> cnt;

```

```
    for (ll x : a) cnt[x]++;

    // 按 key 有序输出
    for (auto &kv : cnt) {
        cout << kv.first << " " << kv.second << "\n";
    }
    return 0;
}
```